



Par Xabi Martinez, le 20/11/2025

Contexte et objectifs du projet

- Projet : backend en Rust
- Objectifs : modéliser, sécuriser, exposer, tester, déployer
- Architecture API REST + PostgreSQL

Contraintes du projet

- Reconversion + activité professionnelle
- Gestion stricte du temps
- Rust : courbe d'apprentissage élevée
- Approche MVP puis itérations
- Documentation rigoureuse

Architecture globale

- API REST Axum
- PostgreSQL + SQLx
- Dual JWT
- Utilisateur $\Leftrightarrow$ API $\Leftrightarrow$ Base de données
- Build Docker multi-stage

Conception Merise et contraintes

- MCD : Users, Pokémon, User_Pokemon
- Relation $N \leq N$ capturée
- Attribut dérivé caught
- Contraintes UNIQUE + cascade
- Modèle stable & normalisé

Sécurité de l'API

- JWT access court / refresh long
- Cookies HttpOnly, SameSite Lax/Strict
- Argon2 pour les mots de passe
- Middlewares Axum
- Mitigation CSRF (cookies + header)

Flux applicatifs

- Login : validation + tokens
- Refresh automatique
- Listing Pokémon
- Capture (idempotente)
- Vérifications & permissions

Extraits de code

```
catch function

pub async fn catch(
    CurrentUser(user_id): CurrentUser,
    State(pool): State<PgPool>,
    Json(payload): Json<CatchByNamePayload>,
) → ApiResponse<(axum::http::StatusCode, String)> {
    let pokemon_id = sqlx::query_scalar::<_, i32>(r#"SELECT id FROM pokemon WHERE name = $1"#)
        .bind(&payload.name)
        .fetch_optional(&pool)
        .await
        .map_err(to_500)?
        .ok_or_else(|| not_found("Pokémon introuvable."))?;

    let _ = sqlx::query(
        r#"
        INSERT INTO user_pokemon (user_id, pokemon_id, nickname)
        VALUES ($1, $2, $3)
        ON CONFLICT (user_id, pokemon_id) DO NOTHING
        "#,
    )
        .bind(user_id)
        .bind(pokemon_id)
        .bind(payload.nickname)
        .execute(&pool)
        .await
        .map_err(to_500)?;

    created("Pokémon marqué comme capturé.")
}
```

```
list_all function

pub async fn list_all(
    CurrentUser(user_id): CurrentUser,
    State(pool): State<PgPool>,
) → ApiResponse<Json<Vec<PokemonWithCaught>>> {
    let rows = sqlx::query_as::<_, PokemonWithCaught>(
        r#"
        SELECT
            p.id          AS id,
            p.name         AS name,
            p.type1        AS type1,
            p.type2        AS type2,
            p.dex_no       AS dex_no,
            p.image_url    AS image_url,
            EXISTS (
                SELECT 1 FROM user_pokemon up
                WHERE up.user_id = $1 AND up.pokemon_id = p.id
            )              AS caught
        FROM pokemon p
        ORDER BY p.id
        "#,
    )
        .bind(user_id)
        .fetch_all(&pool)
        .await
        .map_err(to_500)?;

    Ok(Json(rows))
}
```


Performance et optimisations

- Rust + Tokio
- Requêtes compilées SQLx
- Démarrage : retry DB
- Idempotence et LIMIT
- Calcul dynamique caught

Qualité et tests

- Tests d'intégration (request)
- nextest : exécution rapide
- Linter clippy
- Séparation clean du code
- Documentation claire

Déploiement professionnel

- Docker multi-stage Rust
- docker-compose (API + DB)
- Images légères
- Variables d'environnement
- Tunnel HTTPS possible

Compétences RNCP couvertes

- Modélisation Merise & SQL relationnel
- Développement d'API REST sécurisée
- Sécurité : hachage, JWT, cookies
- Tests, conteneurisation, déploiement

Conclusion

- Projet complet & opérationnel
- Rust : fiabilité & rigueur
- Approche professionnelle
- Sécurité & qualité au cœur

Merci pour votre attention !